

ENSIIE – Programmation Parallèle

RATTRAPAGE 2024 – 2h

I - Question de cours

Question 1) En MPI, à quel type de communication associe-t-on le sigle « PSCW » ? Que signifie-t-il ? À quoi fait-il référence ?

Question 2) Quelles sont les différences entre des opérations non-bloquantes et persistantes ? Lesquelles sont les plus intéressantes en termes de performance ?

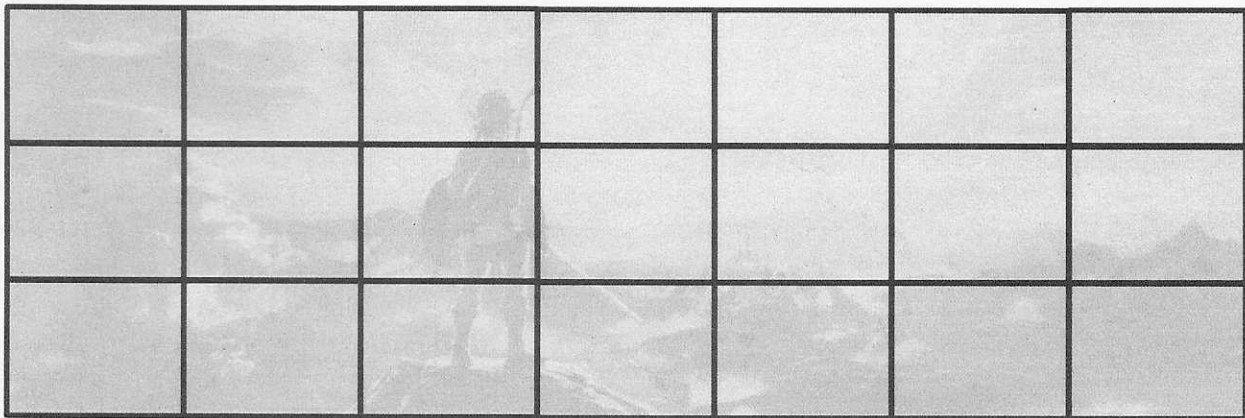
Question 2) Quelles sont les différences entre les communications point-à-point et les communications one-sided (ou rma) ?

Question 4) Que permet de faire le mot clé « MPI_ANY_SOURCE » ? et « MPI_ANY_TAG » ? Dans quel contexte s'utilisent-ils ?

Question 5) Expliquer le fonctionnement de la fonction MPI_Compare_and_swap. Vous pouvez vous servir de schémas.

II - Topologie

On considère une image que l'on découpe comme ci-dessous sur 21 processus MPI.



Pour rappel, on appelle « mailles fantômes » les cases vides supplémentaires, que l'on alloue autour des frontières d'un découpage de maillage, ajoutée pour recevoir les données envoyées par les processus MPI voisins et nécessaires pour réaliser les calculs du processus MPI courant.

- 1) En utilisant les appels MPI de votre choix, réalisez les échanges de mailles fantômes entre les processus MPI dans le cas général (i.e., nous allons considérer uniquement les processus MPI du « milieu », sans conditions particulières aux bords). Dans ce découpage, nous considérons que chaque processus MPI contient une sous-partie de l'image d'une taille de $xel * yel$ éléments de type *pixel*.
- 2) Construisez la topologie cartésienne utilisée dans cet exemple.
- 3) En utilisant la topologie cartésienne créée ci-dessus et des opérations collectives *neighborhood*, réalisez les échanges de mailles fantômes décrits dans la question II-1).

III – MPI I/O et views

Nous considérons dans cette partie que nous avons 4 processus MPI. Ces quatre processus MPI vont écrire dans le même fichier en utilisant l'interface MPI I/O.

Voici les buffers d'entrée de chacun des processus MPI :

P0:

R	A	T	T	R	A	P	A	G	E
---	---	---	---	---	---	---	---	---	---

 P1:

P	R	P	A
---	---	---	---

 P2:

E	D	I	T	I	O	N
---	---	---	---	---	---	---

 P3:

2	0	2	4
---	---	---	---

Donner le code permettant de créer les views et d'écrire dans le fichier commun pour obtenir le contenu suivant :

R	A	E	T	T	2	D	0	R	A	I	P	A	P	T	R	G	E	I	P	A	2	0	4			N
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	---

IV – Correct/Pas correct

Pour les exemples suivants, dire si le code est correct ou non. Si le code n'est pas correct, expliquer pourquoi.

a)

```
MPI_Request reqs[2];
Init(buf1,buf2); // initialisation de buf1 et buf2 sur tous les processus MPI. Buf1 et buf2 contiennent des
valeurs différentes. Le contenu de buf1 et buf2 sont identiques sur les différents processus MPI.
switch(rank) {
    case 0:
        MPI_Isend(buf1, count, type, 1, tag1, comm, &req[0]);
        MPI_Isend(buf2, count, type, 1, tag2, comm, &req[1]);
        MPI_Wait(&req[1]);
        Sleep(1); // attente de 1s
        MPI_Wait(&req[0]);
    break;
    case 1:
        MPI_Recv(buf3, count, type, 0, MPI_ANY_TAG, comm, status);
        MPI_Recv(buf4, count, type, 0, MPI_ANY_TAG, comm, status);
        if(isequal(buf2,buf3)==true) return MPI_SUCCESS;
        else return ERROR;
    break;
}
```

b)

```
switch(rank) {
    case 0:
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Send(buf2, count, type, 1, tag, comm);
    break;
    case 1:
        MPI_Recv(buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Recv(buf2, count, type, MPI_ANY_SOURCE, tag, comm, status);
    break;
    case 2:
        MPI_Send(buf2, count, type, 1, tag, comm);
        MPI_Bcast(buf1, count, type, 0, comm);
    break;
}
```

c) On considère ici un exemple avec 3 processus MPI

```
switch(rank) {
    case 0:
        MPI_Allreduce(sbuf1, rbuf1, count, dtype, MPI_SUM, comm0);
        MPI_Allreduce(sbuf3, rbuf3, count, dtype, MPI_SUM, comm2);
    break;
    case 1:
        MPI_Allreduce(sbuf1, rbuf1, count, dtype, MPI_SUM, comm1);
        MPI_Allreduce(sbuf2, rbuf2, count, dtype, MPI_SUM, comm0);
    break;
    case 2:
        MPI_Allreduce(sbuf2, rbuf2, count, dtype, MPI_SUM, comm2);
        MPI_Allreduce(sbuf3, rbuf3, count, dtype, MPI_SUM, comm1);
    break;
}
```